T3000 API & Protocol

Settings Field Mapping: C++ ↔ Frontend · built Nov 13, 2026

Settings Field Mapping: C++ ↔ Frontend

Overview

This document maps the C++ `Str_Setting_Info` structure (400 bytes) to frontend React form fields.

C++ Source: `T3000-Source/T3000/CM5/ud_str.h` (lines 826-914) **Frontend Interface:** `src/t3-react/features/settings/services/settingsRefreshApi.ts`

Data Flow

TEXT

```
Device (400 bytes)
    ↓ FFI Action 17 (GET_WEBVIEW_LIST)
Raw Byte Array [0-399]
    ↓ parseSettingsData()
DeviceSettings Object
    ↓ UI Binding
React Form Fields
    ↓ User Edit & Save
DeviceSettings Object
    ↓ serializeSettingsData()
Raw Byte Array [0-399]
    ↓ FFI Action 16 (UPDATE_WEBVIEW_LIST)
Device (400 bytes)
```

Field Mapping Table

Tab 1: Basic Information

C++ Field	Offset	Type	Size	Frontend Field	Form Control	Notes
`ip_addr[4]`	0-3	uint8[]	4	`ip_addr`	Input (IP)	Format: "192.168.0.144"
`subnet[4]`	4-7	uint8[]	4	`subnet`	Input (IP)	Format: "255.255.255.0"
`gate_addr[4]`	8-11	uint8[]	4	`gate_addr`	Input (IP)	Format: "192.168.0.0"
`mac_addr[6]`	12-17	uint8[]	6	`mac_addr`	Input (MAC)	Format: "00:14:C6:F2:E0:C3"
`tcp_type`	18	uint8	1	`tcp_type`	Switch	0=DHCP, 1=STATIC
`mini_type`	19	uint8	1	`mini_type`	Dropdown	Device type
`panel_type`	51	uint8	1	`panel_type`	Dropdown	Panel type
`panel_name[20]`	52-71	char[]	20	`panel_name`	Input	Max 20 chars
`en_panel_name`	72	uint8	1	`en_panel_name`	Switch	0=disabled, 1=enabled
`panel_number`	73	uint8	1	`panel_number`	Input (number)	Panel ID
`n_serial_number`	151-154	uint32	4	`n_serial_number`	Input (readonly)	Device serial
`object_instance`	186-189	uint32	4	`object_instance`	Input (number)	BACnet instance

Tab 2: Communication

C++ Field	Offset	Type	Size	Frontend Field	Form Control	Notes
`com0_config`	38	uint8	1	`com0_config`	Dropdown	COM0 mode
`com1_config`	39	uint8	1	`com1_config`	Dropdown	COM1 mode
`com2_config`	40	uint8	1	`com2_config`	Dropdown	COM2 mode
`com_baudrate0`	44	uint8	1	`com_baudrate0`	Dropdown	Baud rate index
`com_baudrate1`	45	uint8	1	`com_baudrate1`	Dropdown	Baud rate index
`com_baudrate2`	46	uint8	1	`com_baudrate2`	Dropdown	Baud rate index
`uart_parity[3]`	235-237	uint8[]	3	`uart_parity`	Dropdown[]	0=None, 1=Odd, 2=Even
`uart_stopbit[3]`	238-240	uint8[]	3	`uart_stopbit`	Dropdown[]	0=1bit, 1=2bits
`network_number`	50	uint8	1	`network_number`	Input (number)	Low byte
`network_number_hi`	252	uint8	1	`network_number_hi`	Input (number)	High byte
`mstp_network_number`	179-180	uint16	2	`mstp_network_number`	Input (number)	MSTP network
`mstp_id`	230	uint8	1	`mstp_id`	Input (number)	MSTP ID
`max_master`	233	uint8	1	`max_master`	Input (number)	Max master
`modbus_port`	183-184	uint16	2	`modbus_port`	Input (number)	Modbus port
`modbus_id`	185	uint8	1	`modbus_id`	Input (number)	Modbus ID
`BBMD_EN`	181	uint8	1	`BBMD_EN`	Switch	BBMD enable
`fix_com_config`	257	uint8	1	`fix_com_config`	Input (number)	Fixed COM config

Tab 3: Time Settings

C++ Field	Offset	Type	Size	Frontend Field	Form Control	Notes
`time_zone`	149-150	int16	2	`time_zone`	Dropdown	Signed offset
`time_update_since_1970`	190-193	uint32	4	`time_update_since_1970`	Display	Unix timestamp
`en_sntp`	148	uint8	1	`en_sntp`	Switch	0=no, 1=disable, 2=enable
`sntp_server[30]`	195-224	char[]	30	`sntp_server`	Input	SNTP server address
`time_zone_summer_daytime`	194	uint8	1	`time_zone_summer_daytime`	Switch	DST enable
`time_sync_auto_manual`	228	uint8	1	`time_sync_auto_manual`	Switch	0=SNTP, 1=Manual
`start_month`	248	uint8	1	`start_month`	Dropdown	DST start month
`start_day`	249	uint8	1	`start_day`	Input (number)	DST start day
`end_month`	250	uint8	1	`end_month`	Dropdown	DST end month
`end_day`	251	uint8	1	`end_day`	Input (number)	DST end day

Tab 4: DynDNS Settings

C++ Field	Offset	Type	Size	Frontend Field	Form Control	Notes
`en_dyndns`	144	uint8	1	`en_dyndns`	Switch	0=no, 1=disable, 2=enable
`dyndns_provider`	145	uint8	1	`dyndns_provider`	Dropdown	0=3322.org, 1=dyndns.com, 2=no-ip.com
`dyndns_user[20]`	74-93	char[]	20	`dyndns_user`	Input	Username
`dyndns_pass[20]`	94-113	char[]	20	`dyndns_pass`	Input (password)	Password
`dyndns_domain[30]`	114-143	char[]	30	`dyndns_domain`	Input	Domain name
`dyndns_update_time`	146-147	uint16	2	`dyndns_update_time`	Input (number)	Update interval (minutes)

Tab 5: Email Alarms

Email alarm settings are stored in a separate table/structure - not part of Str_Setting_Info

Tab 6: User Login

C++ Field	Offset	Type	Size	Frontend Field	Form Control	Notes
`user_name`	47	uint8	1	`user_name`	Switch	0=no, 1=disable, 2=enable

Tab 7: Expansion IO

Expansion IO settings are stored in a separate table - not part of Str_Setting_Info

---

Readonly/Metadata Fields

C++ Field	Offset	Type	Size	Frontend Field	Usage
`debug`	20	uint8	1	`debug`	Debug flag (internal)
`refresh_flash_timer`	41	uint8	1	`refresh_flash_timer`	Flash refresh (internal)
`en_plug_n_play`	42	uint8	1	`en_plug_n_play`	PnP enable (internal)
`reset_default`	43	uint8	1	`reset_default`	Reset flag (write 88)
`custmer_unite`	48	uint8	1	`custmer_unite`	Customer units
`usb_mode`	49	uint8	1	`usb_mode`	0=device, 1=host
`sd_exist`	182	uint8	1	`sd_exist`	SD card status
`zegbee_exsit`	225	uint8	1	`zegbee_exsit`	Zigbee exists
`LCD_Display`	226	uint8	1	`LCD_Display`	LCD on/off
`zigbee_panid`	231-232	uint16	2	`zigbee_panid`	Zigbee PAN ID
`special_flag`	234	uint8	1	`special_flag`	Special flags
`webview_json_flash`	253	uint8	1	`webview_json_flash`	JSON format flag
`max_var`	254	uint8	1	`max_var`	Max variables (ESP32)
`max_in`	255	uint8	1	`max_in`	Max inputs (ESP32)
`max_out`	256	uint8	1	`max_out`	Max outputs (ESP32)
`write_flash`	258	uint8	1	`write_flash`	Write flash flag

Data Type Conversions

IP Address (4 bytes → string)

TYPESCRIPT

```typescript
// C++: uint8_t ip_addr[4] = {192, 168, 0, 144}
// Frontend: "192.168.0.144"
const bytesToIP = (bytes: number[]): string => bytes.slice(0, 4).join('.');
const ipToBytes = (ip: string): number[] => ip.split('.').map(Number);
```

MAC Address (6 bytes → string)

TYPESCRIPT

```typescript
// C++: uint8_t mac_addr[6] = {0x00, 0x14, 0xC6, 0xF2, 0xE0, 0xC3}
// Frontend: "00:14:C6:F2:E0:C3"
const bytesToMAC = (bytes: number[]): string =>
  bytes.slice(0, 6).map(b => b.toString(16).padStart(2, '0').toUpperCase()).join(':');
const macToBytes = (mac: string): number[] =>
  mac.split(':').map(hex => parseInt(hex, 16));
```

String (char[] → string)

TYPESCRIPT

```typescript
// C++: char panel_name[20] = "Fandu144-BB"
// Frontend: "Fandu144-BB"
const bytesToString = (bytes: number[]): string =>
  String.fromCharCode(...bytes.filter(b => b !== 0));
const stringToBytes = (str: string, maxLen: number): number[] => {
  const bytes = new Array(maxLen).fill(0);
  for (let i = 0; i < Math.min(str.length, maxLen); i++) {
    bytes[i] = str.charCodeAt(i);
  }
  return bytes;
};
```

Multi-byte Numbers (Little Endian)

TYPESCRIPT

```typescript
// uint16: 2 bytes
const bytesToUint16 = (bytes: number[], offset: number): number =>
  bytes[offset] | (bytes[offset + 1] << 8);
const uint16ToBytes = (value: number): number[] =>
  [value & 0xFF, (value >> 8) & 0xFF];

// uint32: 4 bytes
const bytesToUint32 = (bytes: number[], offset: number): number =>
  bytes[offset] | (bytes[offset + 1] << 8) | (bytes[offset + 2] << 16) | (bytes[offset + 3]
const uint32ToBytes = (value: number): number[] =>
  [value & 0xFF, (value >> 8) & 0xFF, (value >> 16) & 0xFF, (value >> 24) & 0xFF];

// int16 (signed): 2 bytes
const bytesToInt16 = (bytes: number[], offset: number): number => {
  const value = bytes[offset] | (bytes[offset + 1] << 8);
  return value > 32767 ? value - 65536 : value; // Convert to signed
};
```

Dropdown Value Mappings

COM Config (com0_config, com1_config, com2_config)

TYPESCRIPT

```typescript
const COM_CONFIG_OPTIONS = [
  { value: 0, label: 'Not Used' },
  { value: 1, label: 'BACnet MSTP' },
  { value: 2, label: 'Main Modbus' },
  { value: 3, label: 'Main PTP' },
  { value: 4, label: 'Sub GSM' },
  { value: 5, label: 'Main Zigbee' },
  { value: 6, label: 'Sub Zigbee' },
  { value: 7, label: 'Sub Modbus' },
  { value: 8, label: 'RS232 Meter' },
  { value: 9, label: 'MSTP Master' },
];
```

Baudrate (com_baudrate0/1/2)

TYPESCRIPT

```typescript
const BAUDRATE_OPTIONS = [
  { value: 0, label: '9600' },
  { value: 1, label: '19200' },
  { value: 2, label: '38400' },
  { value: 3, label: '57600' },
  { value: 4, label: '115200' },
  { value: 5, label: '76800' },
];
```

UART Parity

TYPESCRIPT

```typescript
const PARITY_OPTIONS = [
  { value: 0, label: 'None' },
  { value: 1, label: 'Odd' },
  { value: 2, label: 'Even' },
];
```

UART Stop Bits

TYPESCRIPT

```typescript
const STOPBIT_OPTIONS = [
  { value: 0, label: '1 bit' },
  { value: 1, label: '2 bits' },
];
```

DynDNS Provider

TYPESCRIPT

```typescript
const DYNDNS_PROVIDER_OPTIONS = [
  { value: 0, label: 'www.3322.org' },
  { value: 1, label: 'www.dyndns.com' },
  { value: 2, label: 'www.no-ip.com' },
];
```

Example: Complete Field Parsing

TYPESCRIPT

```typescript
// From 400-byte array to DeviceSettings object
const all = response.data.device_data[0].All; // Array of 400 bytes

const settings: DeviceSettings = {
  // Network (bytes 0-18)
  ip_addr: `${all[0]}.${all[1]}.${all[2]}.${all[3]}`,
  subnet: `${all[4]}.${all[5]}.${all[6]}.${all[7]}`,
  gate_addr: `${all[8]}.${all[9]}.${all[10]}.${all[11]}`,
  mac_addr: all.slice(12, 18).map(b => b.toString(16).padStart(2, '0').toUpperCase()).join('
  tcp_type: all[18],

  // Device info
  mini_type: all[19],
  panel_type: all[51],
  panel_name: String.fromCharCode(...all.slice(52, 72).filter(b => b !== 0)),
  en_panel_name: all[72],
  panel_number: all[73],

  // Serial number (4 bytes, little endian, offset 151)
  n_serial_number: all[151] | (all[152] << 8) | (all[153] << 16) | (all[154] << 24),

  // ... continue for all fields
};
```

Update Flow (Save Settings)

TYPESCRIPT

```
// From DeviceSettings object to 400-byte array
const all = new Array(400).fill(0);

// Network settings
const ipParts = settings.ip_addr.split('.').map(Number);
all[0] = ipParts[0];
all[1] = ipParts[1];
all[2] = ipParts[2];
all[3] = ipParts[3];

// MAC address
const macParts = settings.mac_addr.split(':').map(hex => parseInt(hex, 16));
all[12] = macParts[0];
all[13] = macParts[1];
// ... etc

// String fields
const nameBytes = stringToBytes(settings.panel_name, 20);
for (let i = 0; i < 20; i++) {
  all[52 + i] = nameBytes[i];
}

// Multi-byte numbers
const serialBytes = uint32ToBytes(settings.n_serial_number);
all[151] = serialBytes[0];
all[152] = serialBytes[1];
all[153] = serialBytes[2];
all[154] = serialBytes[3];

// Send to device via FFI
POST /api/t3000/ffi/call
{
  "action": 16,
  "panelId": panelId,
  "serialNumber": serialNumber,
  "entryType": 98,
  "entryIndex": 0,
  "All": all // 400-byte array
}
```